IEEE Region 10 Conference on Computer and Communication Systems, September 1990, Hong Kong

Fast Fourier Transform using Linear Tagged Systolic Array

Susanta Sarkar , A. K. Majumdar

Computer Science & Engineering Department
Indian Institute of Technology
Kharagpur 721 302
INDIA

Abstract: A systematic methodology for design of systolic arrays from a set of non-linear and non-uniform recurrence equations is discussed in this paper. A novel architectural idea, which is named Tagged Systolic Array, is introduced. A Tagged Systolic Array uses tags for data routing among processing elements. The design methodology described broadens the class of algorithms amenable for Tagged Systolic Array implementation. The design methodology is illustrated by deriving a tagged systolic array for the Fast Fourier Transform.

I. Introduction

Considerable effort has been directed towards the development of a systematic method for the design of systolic arrays [2,3,4,6,7,11,12,14]. More recently, based on the pioneering work of Karp, Miller & Winograd [10], Quinton [8,9] described a systematic design methodology using Uniform Recurrence Equations (URE). The main feature of an algorithm, that can be represented using a set of URE, is the locality and regularity of its dependence graph (DG) [11]. A transformation technique has also been discussed by Dongen & Quinton [13] so that a set of non-uniform linear recurrence equations can be converted to a set of URE.

When an algorithm cannot be represented using a set of URE, much less is known about the method of deriving a systolic design. Such algorithms can be, in general, expressed using a set of Non-Linear and Non-Uniform Recurrence Equations (NLNURE). Many important algorithms such as Fast Fourier Transform (FFT), Fast Hadamard Transform, Bitonic sorting etc. belong to this class.

In section II of this paper, a systematic design methodology is presented for deriving systolic array implementations of algorithms which can be represented by a set of NLNURE. In the process, a new architectural idea, called the Tagged Systolic Array (TSA), is introduced. In a TSA, tags are attached to the results of a particular computation for properly routing them to other processing elements (PEs) where the result of that particular computation is required. A TSA uses only nearest neighbour local communication links for sending data. The proposed design methodology is illustrated in section III by deriving a tagged systolic array for FFT. Performance of the derived design is analysed in section IV. Section V concludes the paper.

II. Design Methodology

We are interested in algorithms that are represented using a set of m recurrence equations of the form :-

for $1 \le i \le m$,

$$A_i(z) = f_i[A_{i1}(z-u_{i1}),\ldots,A_{ik}(z-u_{ik})] \quad \ldots(1)$$

where $i1,\ldots,ik \in \{1,2,\ldots,m\}$

Let Z denote the set of all integers and Z^n denote the set of n-tuples of integers (Z^n is a subset of n-dimensional Euclidean space). The computations are assumed to be attached to the integral index points $z \in Z^n$. For a system of non-uniform linear recurrence equations of the form (1), we may follow the uniformization technique proposed in [13], according to which, we need to find a set of integral basis vectors $B_1,\ldots,B_t$ such that any dependence vector u_{ij} may be expressed as a non-negative integral combination of these vectors. A linear timing function T is found such that it is compatible with the basis vectors chosen i.e. $\forall Bj : T.Bj>0$. Any other timing function is permissible if it is compatible with the dependence vectors i.e. if computation at z_2 is dependent on computation at z_1, then $T(z_1)$must be less than $T(z_2)$.

For a non-linear and non-uniform system of recurrence equation of the form (1), the dependence vector u_{ij} is a function of z (u_{ij} can be written as $u_{ij}=g(z)$). We assume that the NLNURE (1) cannot be transformed into a system of URE by any known technique. Let θ be the set of all dependence vectors for all the size parameters of a problem. The restriction imposed on the dependence vectors is that a dependence vector $u_{ij} \in \theta$ is required to be a semipositive vector (A vector is called semipositive if none of its co-ordinates are negative and not all of them are zero). However, for a system of recurrence equations of the form (1), the above restriction may not be satisfied. To satisfy the restriction, we look for a suitable reindexing transformation. The restriction that u_{ij} is a semipositive vector allows replacement of non-local dependence vectors with integral combinations of basis vectors (which themselves could be chosen as semipositive vectors) and selection of a permissible linear timing function T. Since a dependence vector u_{ij} is replaced by an integral combination of basis vectors, we can route data along the directions of the basis vectors, the integer coefficient of a basis vector being the distance through which a data moves along that direction.

For deriving a systolic design , we need to allocate the computations attached with integral index points of Z^n

CH2866-2/90/0000-289 $1.00 © 1990 IEEE

onto a fewer number of processing elements (PEs). A permissible allocation function α satisfies the relation $\alpha(z_1) \neq \alpha(z_2)$ if $T(z_1)=T(z_2)$ and $z_1 \neq z_2$. The allocation function must also satisfy the local connectivity property among the PEs. In the more restricted case, we may select a linear α after localization of DG using basis vectors and selection of a linear timing function T. In this case, a linear allocation function is permissible if it satisfies the relation $T^T.\alpha > 0$ [8]. The transformation of the DG using basis vectors (as outlined above) allows a local interconnection among PEs (with properly chosen linear α) and data flow only in a fixed number of directions. However, the source PE where a variable is generated and the destination PE where a variable is used, may not be neighbouring PEs. The location of the destination PE may change with respect to the location of the source as well as with time. In such cases tags (evaluated based on location and time) are attached to the data generated at the source PE for proper routing of data. This type of architecture that uses tags for data routing is called a Tagged Systolic Array (TSA). If u_{ij} is a dependence vector after reindexing transformation (if one is applied) and $z_1+u_{ij}=z_2$, then $\alpha(z_2)-\alpha(z_1)$ gives the expression for the tag to be evaluated at PE $\alpha(z_1)$. We are interested in the case when tag values attached to the (partial) results of computations in a PE are always the same i.e. constants. This will imply no additional computational effort and time for computing tag values. Thus the permissible allocation function α has to satisfy the additional property of constant tag values.

A TSA employs local and regular interconnection among PEs and hence is suitable for an efficient implementation in VLSI. The tag of a data is checked at a PE to determine its further direction of movement or to decide whether the data is required at the PE itself. It may be noted that in the more general case, we may choose T and α to be linear or non-linear permissible functions that satisfy the properties already discussed.

III. A Tagged Systolic Array for FFT

The DG for an N point FFT [5] (we assume $N=2^m$ where m is an integer) is shown in fig. 1. The computations represented by the DG are assumed to be attached to integral index points (as shown in fig. 1) of the two dimensional plane . The FFT computation is completed in (log N) * stages for an N point FFT. The computation represented by the DG can be represented using a set of recurrence equations :-

if $k = 1$, $1 \le i \le N$
then $A(k, i) = x_i$ ...(2)

if $1<k\le(\log N+1)$, $1\le i\le N$, $1\le(i \bmod 2^{k-1})\le 2^{k-2}$
then $A(k, i) = F[A(k-1, i+2^{k-2}), A(k-1, i)]$ ...(3)

if $1<k\le(\log N+1)$, $1\le i\le N$, $(i \bmod 2^{k-1}) = 0$
or $(i \bmod 2^{k-1}) > 2^{k-2}$
then $A(k, i) = F[A(k-1, i), A(k-1, i-2^{k-2})]$ ...(4)

if $k>(\log N + 1)$, $1\le i\le N$
then $X(N - i) = A(k-1, i)$ ...(5)

In the above recurrence equations, the input data are (for N = 8):- $x_1 = x(7)$, $x_2 = x(3)$, $x_3 = x(5)$, $x_4 = x(1)$, $x_5 = x(6)$, $x_6 = x(2)$, $x_7 = x(4)$, $x_8 = x(0)$.

* All logarithms in this paper are base two.

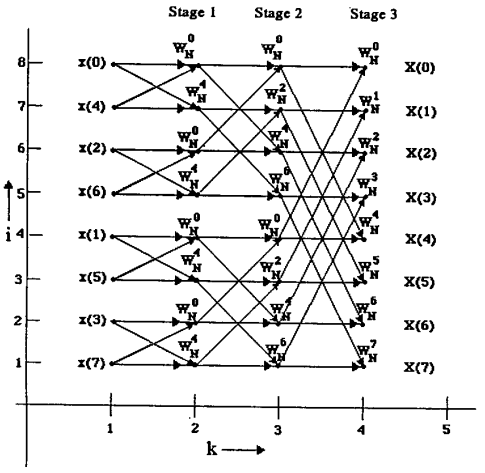

Fig.1 DG for N=8 point FFT

According to the definition of URE given in [13], the above set of recurrence equations can be identified as NLNURE. The set of dependence vectors for equations (3) & (4) (that represent computation at an index point) is $\theta=\{v_1, v_2, v_3\} = \{(1\ 0), (1\ -2^{k-2}), (1\ 2^{k-2})\}$. In this case not all the dependence vectors are semipositive. To make all the dependence vectors semipositive, we reindex the DG such that an index point (k, i) is reindexed to an index point $(k, i+2^{k-1}-1)$.Fig. 2 shows the DG after reindexing.

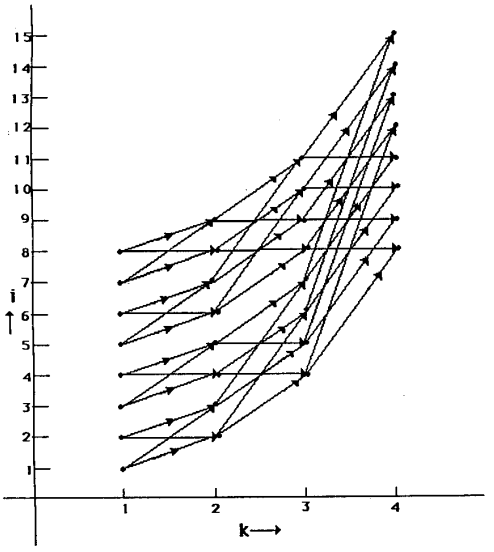

Fig. 2 Reindexed DG

The set of recurrence equations (2) to (5) are modified accordingly :-

if $k=1$, $2^{k-1}\le i\le N+(2^{k-1}-1)$
then $A(k, i) = x_i$ ...(6)

if $1<k\le(\log N+1)$, $2^{k-1}\le i\le N+(2^{k-1}-1)$,
$1\le(i+1-2^{k-1}) \bmod 2^{k-1}\le 2^{k-2}$
then $A(k, i) = F[A(k-1, i), A(k-1, i-2^{k-2})]$ ...(7)

if $1<k\leq(\log N+1)$, $2^{k-1}\leq i\leq N+(2^{k-1}-1)$,
$(i+1-2^{k-1}) \bmod 2^{k-1}=0$ or $(i+1-2^{k-1}) \bmod 2^{k-1}>2^{k-2}$
then $A(k, i) = F[A(k-1, i-2^{k-2}), A(k-1, i-2^{k-1})]$ ...(8)

if $k> (\log N+1)$, $2^{k-1}\leq i\leq N+(2^{k-1}-1)$
then $X(N-(i+1-2^{k-1})) = A(k-1, i-2^{k-2})$ ...(9)

The new set of dependence vectors is $\theta'= \{v'_1, v'_2, v'_3\} = \{(1\ 0), (1\ 2^{k-2}), (1\ 2^{k-1})\}$. The set of basis vectors may be chosen as $\{B_1,B_2\}=\{(1\ 0), (0\ 1)\}$. The linear timing function $T^T=(1\ 1)$ is compatible with the basis vectors chosen. A set of permissible linear allocation functions may now be chosen. However, the designs derived with linear allocation functions have several drawbacks. Hence we try to find out a suitable non-linear allocation function for a better design.

The DG of fig.1 and fig. 2 shows that a positive and a negative half butterfly operation with same data are computed at different index points. Such a pair of interacting computations can be performed simultaneously. The computation of an N point FFT is completed in (log N) stages (fig 1 & fig 2). We note that a single constant is sufficient for the first stage computation of FFT ($w_N^0=-w_N^4$ for $N=8$). Similarly, two constants are required for the second stage computation, four constants are required for the third stage computation and so on. We assume that a PE stores a single constant value and always uses the same constant during the computation of a butterfly. Thus the allocation function should be such that different stages of computations must be allocated to different groups of PEs storing the appropriate constants. The allocation function must also satisfy the requirement of constant tag value attached to the results of computation in a particular PE. Considering all the above factors, we choose the following non-linear permissible allocation function:

for $2\leq k\leq (\log N+1)$, α is such that an index point (k, i) is allocated to PE $\{(i \bmod 2^{k-2})+2^{k-2}\}$

The resulting design shown in fig 3 uses (N-1) PEs. The array is a linear one and uses tags for routing data. Each PE stores only two data (intermediate results) and one constant for computation. Two tag values are also stored in each PE and these values are also constants. A tag is attached to the result of a computation when it is sent to a neighbouring PE. A PE immediately checks the tag of a data after receiving it and stores the data if it is required at the PE otherwise passes the data to the neighbouring PE in the next time step.

The PEs in the pipeline may be divided into different groups that are responsible for computation of different stages of FFT. PEs in the same group always compute butterfly operation in the same time-step. In the design of fig. 3, PE1 performs the computations of stage 1, PE2 & PE3 perform computations of stage 2 and PE4, PE5, PE6 & PE7 are responsible for stage 3 computation.

IV. Performance Analysis

Let us analyse the performance of the (N-1) PE TSA. The array outputs an N point Fourier Transform every N time steps. It has been shown in the next paragraph that a time-step consists of O(log N) elementary cycles. Thus the time performance [1] of the design is O(Nlog N). The latency [11] is O(N). The time performance of a single processor for computation of N point FFT is $O(N\log^2 N)$. The speedup obtained is O(log N). The average processor utilisation of the design presented decreases with the increase in size of the FFT.

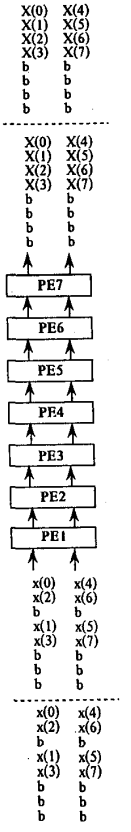

Fig. 3 Tagged Systolic Array for N=8 point FFT

It has been shown in [1] that butterfly operation can be done in O(log N) time using O(log N) area for N point FFT. We note that O(log N) bits are sufficient for proper representation of tag. The additional area required for tag is O(log N). The operations with tags (e.g. decrement and compare) can be done in O(log N) time in O(log N) area. Thus a PE of the TSA for FFT takes O(log N) area and O(log N) time (elementary cycles) for computation. So the total area required for an N point FFT is O(NlogN). It has been shown in [1] that VLSI implementation of butterfly interconnection network takes $O(N^2)$ area. The actual area required may be estimated by taking into account the constant factors involved and it has been estimated (for N=1024 point FFT) that a PE of TSA takes about twice the area that required for a processor employing butterfly interconnection. The chip area required for routing interconnections among PEs in case of TSA is negligible; whereas butterfly interconnection network requires considerable chip area for routing interconnections among PEs because of non-local and non-regular interconnection pattern employed. The total chip area required for butterfly interconnection network may be estimated as approximately double that required for PEs only. Thus the chip area utilisation of TSA is much better than butterfly interconnection network for the computation of FFT especially when N is large. The modularity of the TSA for FFT is another desirable

Table 1 Performance comparison of (N-1) PE TSA for FFT and (NlogN/2) PE network using butterfly interconnection for computation of N point FFT (as discussed in [1]) :-

Factor	(N-1) PE TSA	((N log N) /2) PE FFT network
Area efficiency = computational area / total chip area	1	0.5
Power efficiency = computational power / total power	1	0.5
Design cost	less	comparatively more
Fault tolerance due to interconnect failure	approx. 100% fault tolerant	less than 100%
Modularity	modular	not modular
I/O bandwidth	4 data/ timestep	2N data/ timestep
Processor utilisation	less than 100%	100%
Time performance	0(N log N)	0(log N)
Chip area required	O(N log N)	0 (N^2)

property. Table 1 shows the comparison (based on some performance factors) of TSA and butterfly interconnection networks for the computation of FFT (as discussed in [1]) .

From the analysis done above, we observe that both the designs have certain strong aspects and certain weak aspects. It is difficult to relate all the factors by a common formula which can be utilised as a performance metric. The alternative approach is to assign credit points for each of the factors depending on their relative merit and the total point can be used as a performance index for comparing two designs. If we assign equal credit points for all the factors with equal relative merit, we conclude that the TSA implementation of FFT is better.

V. CONCLUSION

An enhanced systolic architecture viz. Tagged Systolic Array has been presented. A design methodology has been proposed for mapping algorithms onto TSA. The set of recurrence equations describing the algorithm has been taken as the starting point of the design process. Since the recurrence equations in its most general form, i.e non-linear and non-uniform recurrence equations, has been considered, the design methodology presented broadens the class of algorithms that can be explored for a TSA implementation. The linear tagged systolic array derived for FFT has been shown to have many desirable properties for an efficient VLSI implementation.

VI. REFERENCES

[1] C. D. Thompson, "Fourier Transform in VLSI", IEEE Transaction on Computers, vol c-32, no. 11, Nov 1983, p 1047-1057

[2] D. I. Moldovan, "On the Design of Algorithms for VLSI Systolic Arrays" Proceedings of the IEEE, Vol. 71, Jan. 1983, p 113-120.

[3] G. H. Li, B. W. Wah, "The Design of Optimal Systolic Arrays", IEEE Transaction on Computers, Vol. 34, no. 1, 1985, p 66-77.

[4] H. T. Kung, "Why Systolic Architectures?", IEEE Computer, 15(1), Jan. 1983, p 37-46.

[5] H. J. Nussbaumer, "Fast Fourier Transform and Convolution Algorithms", Second. Edition, Springer-Verlag, 1982.

[6] J. D. Ullman, "The Computational Aspects of VLSI", Rockville, MD : Computer Science Press, 1984.

[7] J. M. Delosme, I. C. F. Ipsen, "Efficient Systolic Arrays for the Solution of Toeplitz System: An Illustration of the Methodology for the Construction of Systolic Architectures for VLSI", Systolic Arrays, W. Moore, A. McCabe and R. Urquhart, editors, Adam Hilger, 1986, p 37-46.

[8] P. Quinton, "Automatic Synthesis of Systolic Arrays from Uniform Recurrent Equations", The 11th Annual Symposium on Computer Architecture, Ann Arbor, June 1984, p 208-214.

[9] P. Quinton, "The Systematic Design of Systolic Arrays", IRISA Research Report No. 193, April 1983, p 229-260.

[10] R. M. Karp, R. E. Miller, S. Winogard, "The Organisation of Computations for Uniform Recurrence Equations", JACM, 14(3), July 1967, p 563-590.

[11] S. Y. Kung, "VLSI Array Processor", Prentice Hall, Englewood Cliffs, New Jersey, 1988.

[12] S. K. Rao, "Regular Iterative Algorithms and Their Implementations on Processor Arrays", Ph. D. Thesis, Stanford University, USA, 1985.

[13] V. V. Dongen, P. Quinton, "Uniformization of linear recurrence equations: a Step towards Automatic Synthesis of Systolic Arrays", Proceedings of International Conference on Systolic Arrays, San Diego, California, May 1988.

[14] Y. Yaacoby, P. R. Cappello, "Scheduling a System of Affine Recurrence Equations onto a Systolic Array", Proceedings of International Conference on Systolic Arrays, San Diego, California, May, 1988, p 373-382.